02-12 接口自动化Agent开发实战之Agent工具封装

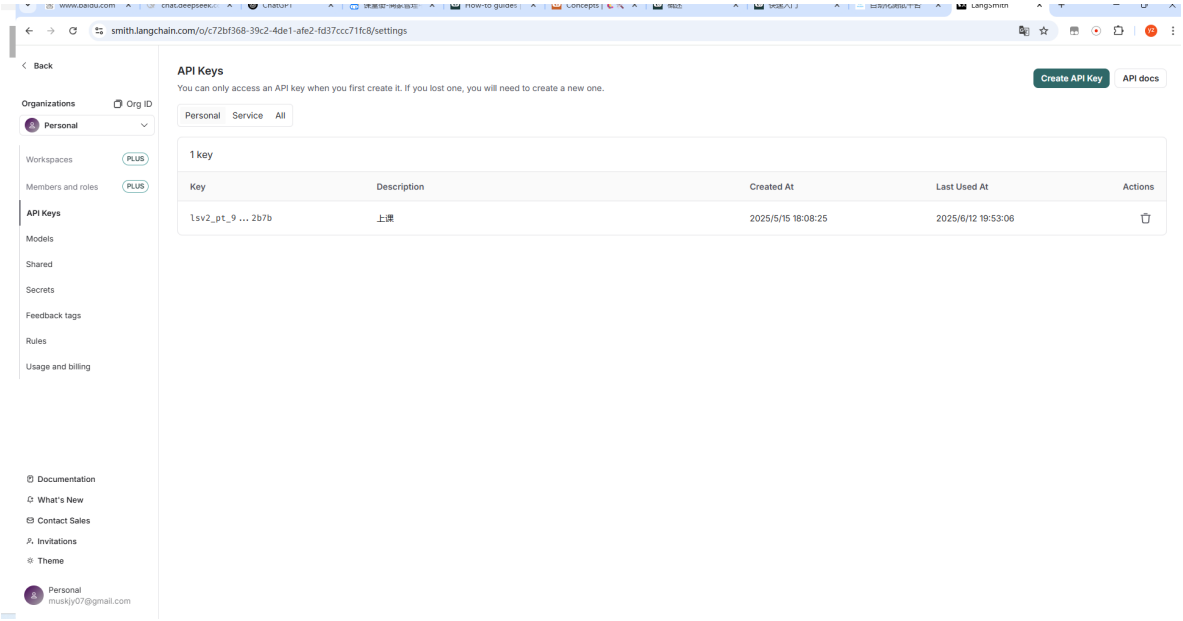
一、langsmith的接入

1、登录注册langsmith

- 地址：https://smith.langchain.com/
- **安装langsmith**

```
pip install  langsmith
```

2、创建apikey



3、在setting中加入

```
# 第1行：设置LangSmith追踪
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = os.getenv("LANG_SMITH_KEY")
os.environ["LANGCHAIN_PROJECT"] = "演示接入langsmith"
```

二、Agent工具封装

1、封装RAG检索工具

- 目标：将 rag/vec_retriever.py 中的检索功能封装为Agent工具
- 输入：用户查询文本、项目名称、检索数量
- 输出：相关接口文档列表
- 文件位置：tools/rag_retrieval_tool.py

代码

```
# @Author  : 木森
# @weixin: python771

from langchain_core.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Optional, Type

from rag.vec_retriever import VectorRetriever


class VectorRetrieverInput(BaseModel):
    """
    RAG检索工具类输入参数
    """
    query: str = Field(description="查询内容")
    k: int = Field(default=3, description="返回结果数量")
    project_name: dict = Field(default=None, description="项目名称")
    search_type: str = Field(default="hybrid", description="检索类型")
    alpha: float = Field(default=0.6, description="混合检索权重")
    collection_name: str = Field(default="webapi", description="向量数据库的集合名称")


class VectorRetrieverTool(BaseTool):
    """
    RAG检索工具类
    """
    name: str = 'vector_retriever'
    description: str = """
    RAG检索工具类，用于向知识库检索相关项目接口的文档信息
    功能：
    - 支持向量语义检索，关键词检索和混合检索
    - 可以按照项目过虑检索结果
    - 返回json形式的接口文档信息

    使用场景：
    - 用户查询特定接口的文档
    - 生成特点接口的测试用例时，需要使用该工具检索出接口的文档

    输入参数：
    - query: 检索查询的文本(必需)
    - k: 返回的数量 ，默认5
    - search_type: 搜索的类型 ,默认是hybrid
    - alpha: 混合检索的权重设置，默认是0.6
    - project_name : 项目名称
    - collection_name : 向量数据库的集合名称，默认是webapi
    """
```

```python
    args_schema: Type[BaseModel] = VectorRetrieverInput

    def _run(self, query: str,
             collection_name: str = "webapi",
             project_name: Optional[str] = None,
             k: int = 5,
             search_type: str = 'hybrid',
             alpha: float = 0.6):
        """正常调用工具的核心执行逻辑"""
        if project_name:
            filters = {"name": "project", "value": project_name}
        else:
            filters = None

        with VectorRetriever(connection_name=collection_name) as vector:
            if search_type == "bm25":
                documents = vector.bm25_invoke(query, k, filters)
            elif search_type == "hybrid":
                documents = vector.hybrid_invoke(query, k, alpha, filters)
            else:
                # 默认使用向量检索
                documents = vector.invoke(query, k, filters)
        #  格式化结果，进行返回
        return self.__format_results(documents.objects)

    def __format_results(self, results):
        """格式化结果，进行返回"""
        # 获取接口的详细字段数据
        result = []
        for item in results:
            result.append(dict(interface_id=str(item.uuid), **item.properties))
            # result.append(Document(
            #     page_content=item.properties['summary'],
            #     metadata=dict(interface_id=str(item.uuid), **item.properties)
            # ))
        return result

    def _arun(self, query: str, k: int, filters: dict, search_type: str, alpha:
float):
        """工具支持异步调用的核心逻辑"""


if __name__ == '__main__':
    tool = VectorRetrieverTool()
    res = tool.invoke({
        "query": "创建项目",
        "project": "web测试平台",
        "k": 1

    })
    print(res)
```

2、封装基础用例生成工具

- 目标：将 workflows/base_case_generator.py 封装为工具
- 输入：接口文档、项目名称、接口ID
- 输出：基础测试用例列表
- 文件位置：tools/base_case_generation_tool.py

代码

```
# @Author  : 木森
# @weixin: python771
"""
生成基础用例的工具

"""

from langchain_core.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Optional, Type, Dict, Any
from workflows.base_case_generator import BaseCaseGeneratorFlow


class BaseCaseGenerationInput(BaseModel):
    """基础用例生成工具的输入参数"""
    api_doc: Dict[str, Any] = Field(description="接口详细文档信息，包含接口的所有元数据")
    project_name: str = Field(description="项目名称，用于获取项目下所有接口信息")
    interface_id: str = Field(description="接口ID，用于存储生成的用例时关联接口")


class BaseCaseGeneratorTool(BaseTool):
    """基础用例生成的工具"""
    name: str = "base_case_generation"
    description: str = """
        基础用例生成工具，用于根据接口文档生成基础测试用例。

        功能：
        - 根据接口文档生成基础测试用例
        - 自动进行覆盖率校验
        - 如果覆盖率不足，自动补充生成用例
        - 自动保存用例到数据库

        工作流程：
        1. 获取项目下所有接口信息
        2. 生成基础测试用例
        3. 进行覆盖率校验
        4. 如果覆盖率不足，补充生成用例
        5. 保存用例到数据库并输出结果

        使用场景：
        - 为单个接口生成完整的基础测试用例
        - 为单个接口生成并进行测试覆盖率分析时
        - 作为结构化用例生成的前置步骤

        输入参数：
        - api_doc: 接口详细文档（必需）
```

```
        - project_name: 项目名称（必需）
        - interface_id: 接口ID（必需）
        """
    args_schema: Type[BaseModel] = BaseCaseGenerationInput

    def _run(self, api_doc: Dict[str, Any], project_name: str, interface_id:
str):
        """
        生成基础测试用例
        :param api_doc: 接口详细文档
        :param project_name: 项目名称
        :param interface_id: 接口ID
        :return:
        """
        workflow = BaseCaseGeneratorFlow().create_workflow()
        state = workflow.invoke({
            "api_doc": api_doc,
            "project_name": project_name,
            "interface_id": interface_id
        })
        return self.__get_generation_report(state)

    def __get_generation_report(self, state):
        """获取用例生成的详细报告"""
        return {
            "coverage_report": state.get("coverage_report", ""),
            "cases": state.get("cases", []),
            "all_api": state.get("all_api", []),
        }


if __name__ == '__main__':
    tool = BaseCaseGeneratorTool()
    result = tool.invoke({
        "api_doc": {'interface_id': 'c76823c7-5983-322b-9212-4348a65f84f6',
'version': None, 'description': '创建项目',
                    'path': '/api/pro/projects/', 'tag': None,
                    'responses': '{"201": {"type": null, "description": ""},
"422": {"type": null, "description": ""}}',
                    'summary': '创建项目', 'method': 'POST', 'module': None,
'requestBody': '{}',
                    'project': 'web测试平台', 'parameters': '[]'},
        "interface_id": "ea4a5cd5-e75c-3b69-aec7-bee41a10e189",
        "project_name": "web测试平台"
    })
    print(result)
```

3、封装单条结构化用例生成工具

- 目标：将 workflows/strutured_case_generation.py 封装为工具
- 输入：基础用例信息、接口文档、测试环境数据
- 输出：结构化测试用例
- 文件位置：tools/structured_case_generation_tool.py

代码

```
# @Author  : 木森
# @weixin: python771
from typing import Optional, Dict, List, Any, Type
from langchain_core.tools import BaseTool
from pydantic import BaseModel, Field
import json
import traceback

from workflows.strutured_case_generation import ApiCaseStructuredGeneratorFlow


class StructuredCaseGenerationInput(BaseModel):
    """结构化用例生成工具的输入参数"""
    case_info: Dict[str, Any] = Field(description="基础用例信息，包含用例名称、步骤、预期结果、依赖等")
    case_api: Dict[str, Any] = Field(description="接口详细文档信息，包含接口的所有元数据")
    project: str = Field(description="项目名称")
    base_case_id: Optional[str] = Field(default=None, description="基础用例ID，用于关联基础用例")
    test_data: Optional[Dict[str, Any]] = Field(default=None, description="测试环境数据，如测试账号、基础URL等")
    additional_info: Optional[str] = Field(default="", description="额外的说明信息")


class OneStructuredCaseGenerationTool(BaseTool):
    """结构化用例生成工具类"""

    name: str = "one_structured_case_generation"
    description: str = """
    结构化用例生成工具，用于将单条的基础用例转换为可执行的结构化测试用例。

    功能：
    - 根据基础用例信息生成结构化测试用例
    - 自动检测和获取前置依赖接口
    - 获取测试环境数据和可用函数
    - 验证生成用例的可用性
    - 支持用例重新生成和优化
    - 自动保存用例到数据库

    工作流程：
    1. 检测用例前置依赖接口
    2. 获取依赖接口详细信息（如有）
    3. 获取测试环境数据
    4. 生成结构化用例
    5. 验证用例可用性
    6. 如果验证失败，重新生成（最多3次）
    7. 保存用例到数据库

    使用场景：
    - 将基础用例转换为可执行的结构化用例
    - 需要生成包含具体请求参数和断言的测试用例
    - 自动化测试用例的生成和验证
```

```
    输入参数:
    - case_info: 基础用例信息（必需）
    - case_api: 接口文档（必需）
    - project: 项目名称（必需）
    - base_case_id: 基础用例ID（可选）
    - test_data: 测试环境数据（可选）
    - additional_info: 额外说明信息（可选）
    """

    args_schema: Type[BaseModel] = StructuredCaseGenerationInput

    def _run(self,
             case_info: Dict[str, Any],
             case_api: Dict[str, Any],
             project: str,
             base_case_id: Optional[str] = None,
             test_data: Optional[Dict[str, Any]] = None,
             additional_info: Optional[str] = ""):
        """根据基础用例，生成结构化用例"""
        workflow = ApiCaseStructuredGeneratorFlow().create_workflow()
        state = workflow.invoke({
            "case_info": case_info,
            "case_api": case_api,
            "project": project,
            "base_case_id": base_case_id,
            "test_data": test_data,
            "additional_info": additional_info
        })
        return state.get('output_case', {})


if __name__ == '__main__':
    tool = OneStructuredCaseGenerationTool()
    case_info = {'name': '正常创建项目-参数正确',
                 'steps': ['调用用户登录接口获取有效token', '发送POST请求
到/api/pro/projects/',
                           '请求体包含正确的name和user参数'],
                 'expected': ['HTTP状态码为201', '响应体包含id、name、user_id和
create_time字段', 'name值与请求参数一致'],
                 'dependencies': ['用户登录']}
    case_api = {'version': None, 'description': '创建项目', 'path':
'/api/pro/projects/', 'tag': None,
                'responses': '{"201": {"id": {"type": "integer", "description":
"项目id"}, "name": {"type": "string", "description": "项目名称"}, "user_id":
{"type": "integer", "description": "项目负责人"}, "create_time": {"type":
"string", "description": "创建时间"}}, "422": {"detail": {"type": "array",
"description": ""}}}',
                'summary': '创建项目', 'method': 'POST', 'module': None,
                'requestBody': '{"name": {"type": "string", "description": "项目
名称"}, "user": {"type": "integer", "description": "项目负责人"}}',
                'project': 'web测试平台', 'parameters': '[]', 'distance':
-3.5762786865234375e-07}
    project = 'web测试平台'
    tool.invoke({"case_info": case_info,
                 "case_api": case_api,
                 "project": project,
                 "base_case_id": "d0531189-c416-46a2-9285-becc9c01cd7c",
                 "test_data": {
```

```
                    "username": "python771",
                    "password": "musen12399",
                    "base_url": "http://124.222.144.182:8888"
                }}, )
```

4、封装批量结构化用例生成工具

- 目标：将 workflows/all_case_strutured_graph.py 封装为工具
- 输入：基础用例列表、接口文档、项目信息
- 输出：批量结构化用例列表
- 文件位置：tools/batch_structured_case_tool.py

代码

```
# @Author  : 木森
# @weixin: python771
"""
批量结构化用例生成工具
将批量结构化用例生成工作流封装为Agent工具
"""

from typing import Optional, Dict, List, Any, Type
from langchain_core.tools import BaseTool
from pydantic import BaseModel, Field

from workflows.all_case_strutured_graph import StructuredCaseGeneratorFlow


class BatchStructuredCaseInput(BaseModel):
    """批量结构化用例生成工具的输入参数"""
    base_cases: List[Dict[str, Any]] = Field(description="基础用例列表，每个用例包含名称、步骤、预期结果、依赖等信息")
    api_info: Dict[str, Any] = Field(description="接口详细文档信息，包含接口的所有元数据")
    project_name: str = Field(description="项目名称")
    test_data: Optional[Dict[str, Any]] = Field(default=None, description="测试环境数据，如测试账号、基础URL等")
    additional_info: Optional[str] = Field(default="", description="用例生成的额外说明信息")


class BatchStructuredCaseTool(BaseTool):
    """批量结构化用例生成工具类"""

    name: str = "batch_structured_case_generation"
    description: str = """
    批量结构化用例生成工具，用于并行处理多个基础用例，将它们转换为可执行的结构化测试用例。

    功能：
    - 并行处理多个基础用例
    - 为每个基础用例生成对应的结构化测试用例
    - 自动检测和获取前置依赖接口
    - 验证生成用例的可用性
```

```
    - 支持用例重新生成和优化
    - 自动保存用例到数据库

    工作流程：
    1. 任务拆分：将基础用例列表拆分为并行任务
    2. 并行生成：为每个基础用例并行生成结构化用例
    3. 结果汇总：收集所有生成的结构化用例

    使用场景：
    - 批量处理多个基础用例
    - 需要高效生成大量结构化用例时
    - 接口测试用例的批量自动化生成

    输入参数：
    - base_cases: 基础用例列表（必需）
    - api_info: 接口文档（必需）
    - project_name: 项目名称（必需）
    - test_data: 测试环境数据（可选）
    - additional_info: 额外说明信息（可选）
    """

    args_schema: Type[BaseModel] = BatchStructuredCaseInput

    def _run(self,
             base_cases: List[Dict[str, Any]],
             api_info: Dict[str, Any],
             project_name: str,
             test_data: Optional[Dict[str, Any]] = None,
             additional_info: Optional[str] = "") -> str:
        """
        执行批量结构化用例生成

        Args:
            base_cases: 基础用例列表
            api_info: 接口文档
            project_name: 项目名称
            test_data: 测试环境数据
            additional_info: 额外说明信息

        Returns:
            格式化的生成结果JSON字符串
        """
        # 直接使用现有的工作流
        workflow = StructuredCaseGeneratorFlow()
        app = workflow.create_workflow()

        # 准备输入状态
        input_state = {
            "base_cases": base_cases,
            "api_info": api_info,
            "project_name": project_name,
            "test_data": test_data or {},
            "additional_info": additional_info or ""
        }

        # 执行工作流
        state = app.invoke(input_state)
        return state.get("structured_cases", [])
```

5、封装完整用例生成工具

- 目标：将 workflows/compete_case_generation.py 封装为工具
- 输入：接口文档、项目名称、测试环境配置
- 输出：完整的测试用例集（基础+结构化）
- 文件位置：tools/complete_case_generation_tool.py

代码

```
from typing import Type, Dict, Any, Optional

from langchain.tools import BaseTool
from pydantic import BaseModel, Field

from workflows.compete_case_generation import CompleteCaseGenerationFlow


class CompleteCaseGenerationInput(BaseModel):
    """完整用例生成工具的输入参数"""
    api_doc: Dict[str, Any] = Field(
        description="接口文档信息，包含接口的详细描述、参数、响应等信息"
    )
    project_name: str = Field(
        description="项目名称，用于标识用例所属项目"
    )
    interface_id: str = Field(
        description="接口唯一标识ID"
    )
    test_data: Optional[Dict[str, Any]] = Field(
        default=None,
        description="测试环境数据，可选参数，用于用例生成时的数据引用"
    )


class CompleteCaseGenerationTool(BaseTool):
    """
    完整用例生成工具

    该工具封装了从接口文档到完整用例生成的全流程，包括：
    1. 基础用例生成 - 根据接口文档生成基础测试用例
    2. 结构化用例生成 - 将基础用例转换为结构化的可执行用例
    3. 生成完整报告 - 提供用例生成的详细报告

    适用场景：
    - 需要一次性完成从接口文档到可执行用例的完整流程
    - 批量生成项目的所有接口用例
    - 自动化测试用例的快速生成
    """

    name: str = "complete_case_generation"
    description: str = """
    完整用例生成工具 - 从接口文档到完整用例的全流程生成。
```

```
    功能特点：
    1. 全流程自动化：从接口文档直接生成可执行的结构化用例
    2. 智能用例设计：基于接口特征自动设计测试场景
    3. 数据关联处理：支持测试环境数据的自动关联
    4. 质量保证：内置用例验证和优化机制
    5. 详细报告：提供用例生成过程的完整分析报告

    输入参数：
    - api_doc: 接口文档信息（必需）
    - project_name: 项目名称（必需）
    - interface_id: 接口ID（必需）
    - test_data: 测试环境数据（可选）

    输出格式：JSON格式，包含基础用例、结构化用例和生成报告
    """
    args_schema: Type[BaseModel] = CompleteCaseGenerationInput

    def _run(self,
             api_doc: Dict[str, Any],
             project_name: str,
             interface_id: str,
             test_data: Optional[Dict[str, Any]] = None):
        """
        执行完整用例生成流程

        Args:
            api_doc: 接口文档信息
            project_name: 项目名称
            interface_id: 接口ID
            test_data: 测试环境数据

        Returns:
            JSON格式的生成结果
        """
        flow = CompleteCaseGenerationFlow().create_workflow()
        # 准备工作流输入状态
        input_state = {
            "api_doc": api_doc,
            "project_name": project_name,
            "interface_id": interface_id,
            "test_data": test_data or {}
        }

        # 执行完整用例生成工作流
        state = flow.invoke(input_state)
        return state.get("structured_cases")


if __name__ == '__main__':
    tool = CompleteCaseGenerationTool()
    api_info = {'version': None, 'description': '创建项目', 'path':
'/api/pro/projects/', 'tag': None,
                'responses': '{"201": {"id": {"type": "integer", "description":
"项目id"}, "name": {"type": "string", "description": "项目名称"}, "user_id":
{"type": "integer", "description": "项目负责人"}, "create_time": {"type":
"string", "description": "创建时间"}}, "422": {"detail": {"type": "array",
"description": ""}}}',
```

```
                'summary': '创建项目', 'method': 'POST', 'module': None,
                'requestBody': '{"name": {"type": "string", "description": "项目名称"}, "user": {"type": "integer", "description": "项目负责人"}}',
                'project': 'web测试平台', 'parameters': '[]', 'distance': -3.5762786865234375e-07}

    result = tool.invoke({"api_doc": api_info,
                          'project_name': "web测试平台",
                          "interface_id": "ea4a5cd5-e75c-3b69-aec7-bee41a10e189"
                          })
```